

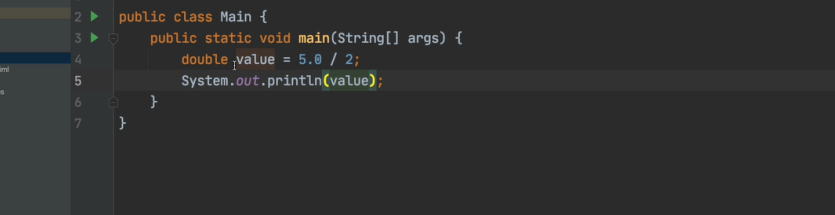
Операции с числами

Введение

В этой теме вы узнаете, что можно делать с числами, об арифметических операциях, операциях сравнения и операциях с присвоением. Также узнаете о сложных операциях, которые можно выполнять с помощью методов специального класса `Math`.

Арифметические операции

Остановимся чуть подробнее на делении.



The screenshot shows an IDE with a project named "NumbersAndDates". The source code in "Main.java" is as follows:

```
1 public class Main {  
2  
3     public static void main(String[] args) {  
4         double value = 5.0 / 2;  
5         System.out.println(value);  
6     }  
7 }
```

The "Run" tab at the bottom shows the command executed:

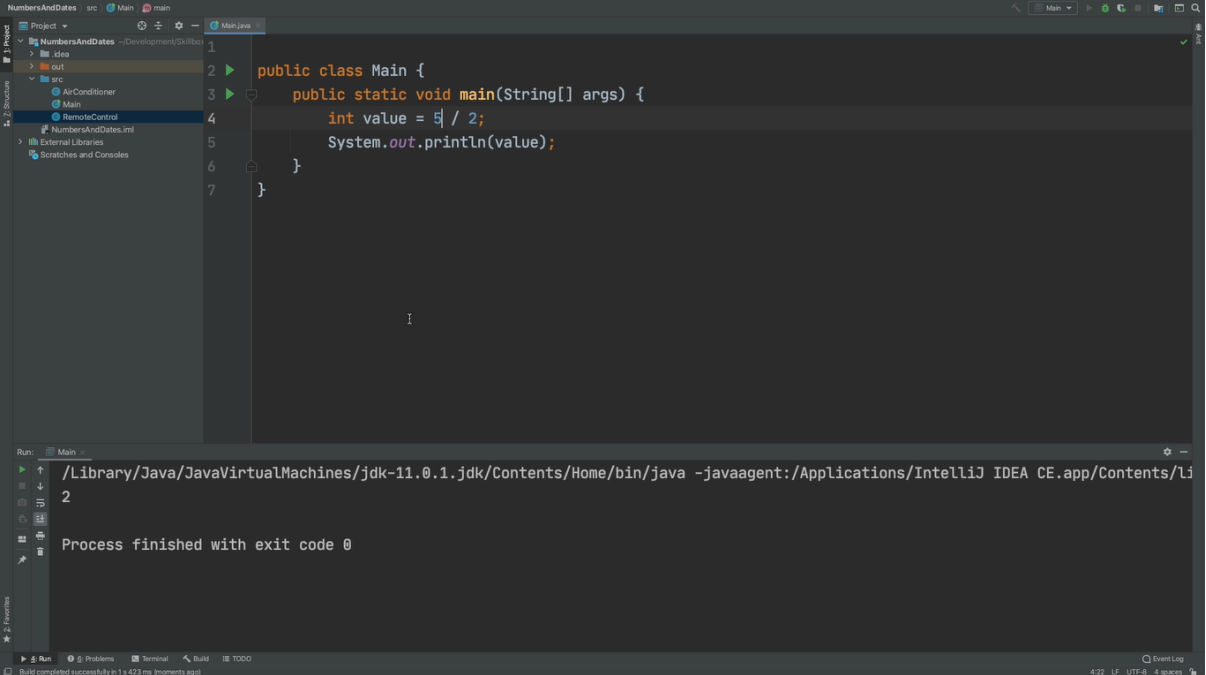
```
/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/lib
```

The output of the program is:

```
2.5
```

Below the output, it states: "Process finished with exit code 0".

Если же вы работаете с целыми числами, то результат, на первый взгляд, может показаться неожиданным — $5 / 2$ будет просто 2:



```
1 public class Main {
2     public static void main(String[] args) {
3         int value = 5 / 2;
4         System.out.println(value);
5     }
6 }
7
```

Run: Main

/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/lib

2

Process finished with exit code 0

Почему так? При делении целых чисел друг на друга получается тоже целое число и не происходит никакого округления. Для простоты запомните, что при делении целого числа на другое целое число нет чисел с плавающей точкой или запятой. Фактически вычисляется, сколько делителей (в данном случае чисел 2) помещается в делимом, то есть в числе 5. Очевидно только 2. Можно запомнить и по-другому: то, что после запятой, отбрасываем. Даже если результат деления 2,999 — получится просто число 2.

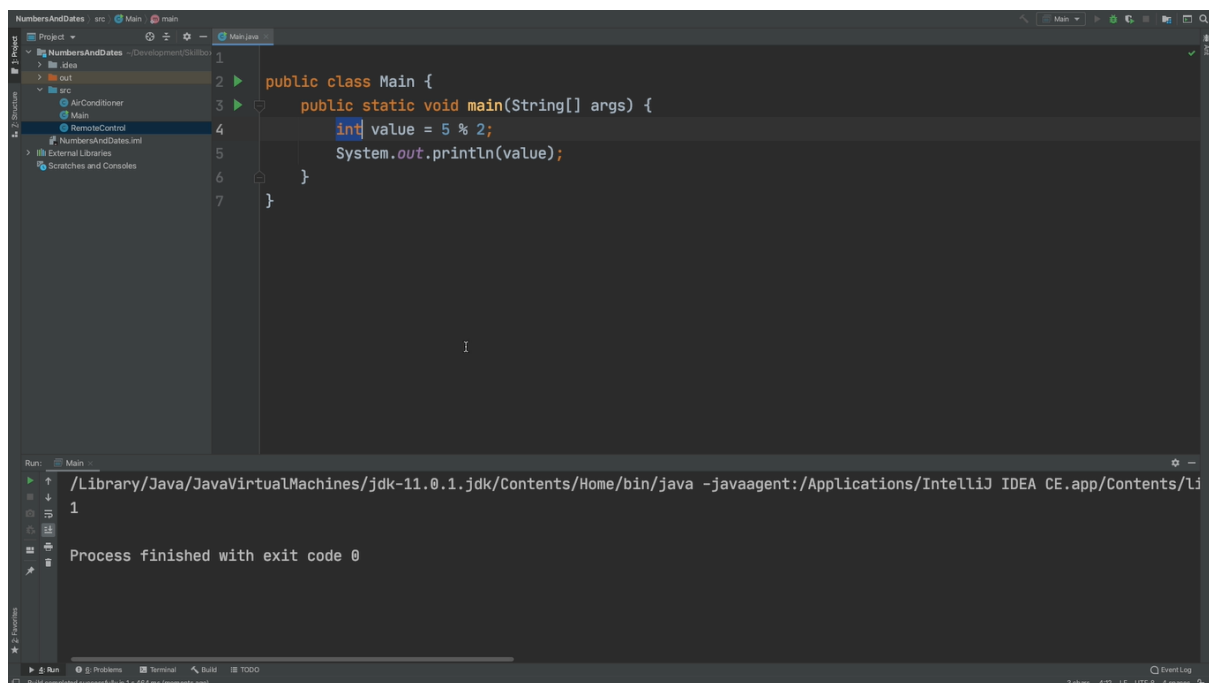
Арифметические операции

```
int sum = 54 + 49;
int diff = 54 - 49;
int mult = 26 * 34;
int div = 35 / 5;
```

О четырёх арифметических операциях вы знаете ещё с начальной школы, и на них мы не будем останавливаться. Отметим лишь, что вокруг операторов “плюс”, “минус”, “разделить” и “умножить” обязательно следует ставить пробелы — это повысит читабельность кода.

Есть ещё одна операция, с которой вы, возможно, не знакомы — это так называемое **деление с остатком**. Эта операция обозначается символом процента (%). В некотором смысле деление с остатком противоположно делению и вычисляет ту часть делимого, которую не удалось нацело разделить на делитель.

В нашем случае, если напишем так, то получим число 1:



The screenshot shows an IDE window with a project named 'NumbersAndDates'. The 'Main.java' file is open, showing the following code:

```
1 public class Main {
2
3     public static void main(String[] args) {
4         int value = 5 % 2;
5         System.out.println(value);
6     }
7 }
```

The code is executed, and the output is shown in the 'Run' console at the bottom:

```
Run: Main
/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/Lj
1
Process finished with exit code 0
```

Почему 1? Потому что если 5 разделить на 2, то нацело делится только число 4. Оставшееся число 1 нацело не делится — это и есть остаток от деления.

Рассмотрим ещё пример. Если нужно вычислить остаток от деления числа 11 на 3, то число 11 условно делим на две части: ту, которая делится нацело на 3 (число 9), и ту, которая не делится нацело на 3 (число 2). Соответственно, остаток от деления — число 2.

Остаток от деления

11 % 3

остаток
от деления

↓

$$11 / 3 = 9 / 3 + 2 / 3$$

деление
нацело нацело
 не делится

Из арифметических операций можно составлять целое выражение, можно использовать круглые скобки — всё как в обычной математике.

03:10–05:06

Операции с присвоением

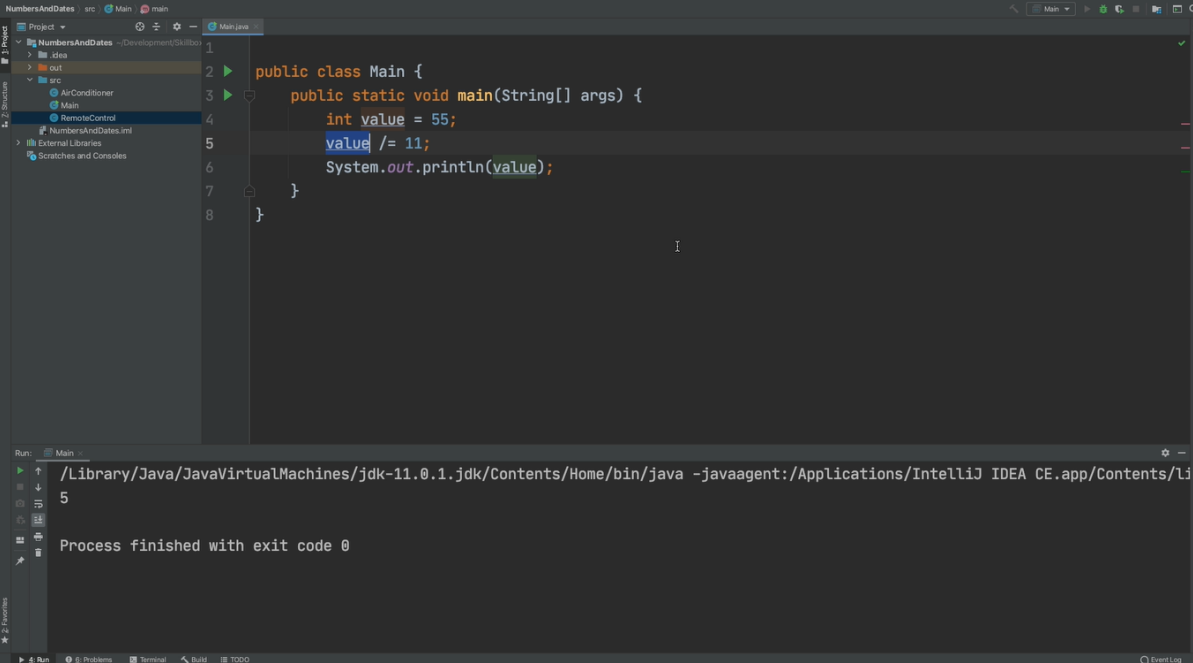
Есть ещё один тип операций с числами — так называемые операции с присвоением, и он существует для всех арифметических операций. Например, если вы хотите увеличить переменную `value` на некоторое число, вы можете написать `value = value + это число`. Есть более краткая запись: `+=`. Она сокращает количество кода, который пишется для такого увеличения.

Операции с присвоением

```
int value = 54;  
value = value + 10;  
  
value += 10;
```

Посмотрим примеры операций с присвоением в коде.

Есть некоторое число `value = 55`. Напишем операцию деления с присвоением `/=` и поставим число 11. Сначала произойдёт деление 55 на 11, получится число 5, и это число 5 будет присвоено переменной `value`. Запускаем код и получаем число 5:



The screenshot shows an IDE window with a project named 'NumbersAndDates'. The 'Main.java' file is open, showing the following code:

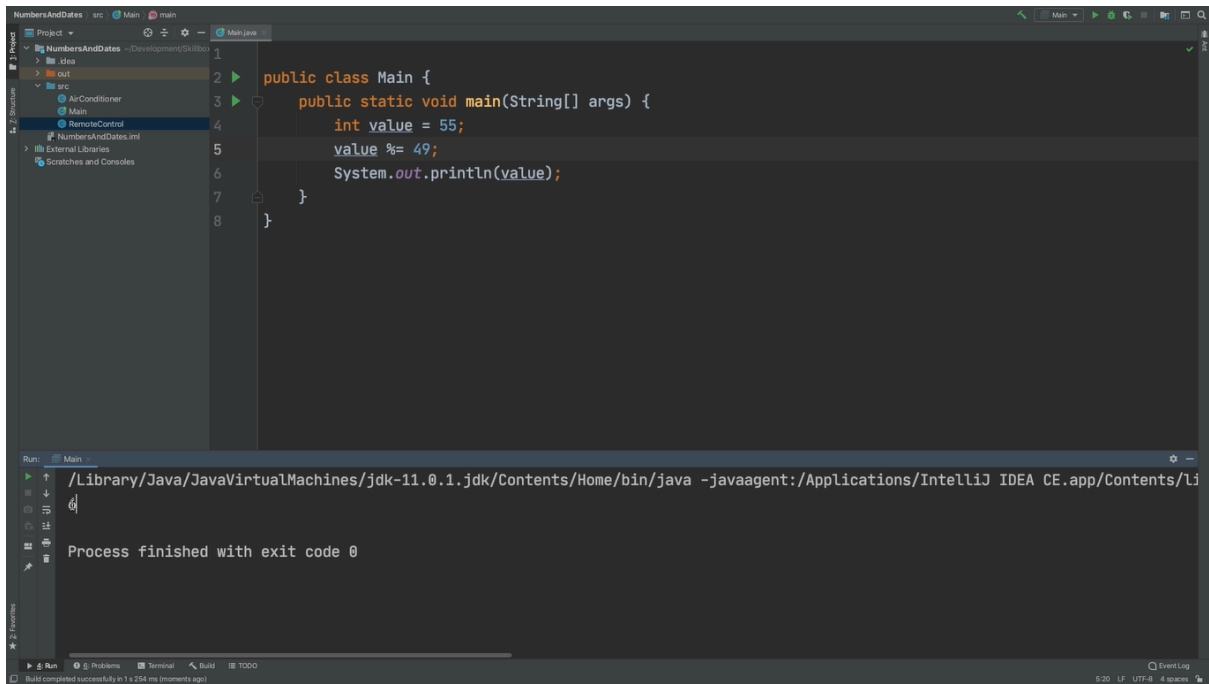
```
1 public class Main {
2     public static void main(String[] args) {
3         int value = 55;
4         value /= 11;
5         System.out.println(value);
6     }
7 }
8
```

The 'Run' button is highlighted. Below the code editor, the 'Run' console shows the execution path and the output:

```
Run: Main
/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/Lj
5
Process finished with exit code 0
```

То есть `value /= 11` — это то же самое, как если бы вы написали `value = value / 11`.

Если нужно посчитать остаток от деления, то всё произойдёт в той же последовательности, только знак деления поменяем на процент:



05:06–07:39

Операции сравнения

Ещё один важный тип операций, которые можно производить с числами, — это операции сравнения.

Операторов сравнения всего шесть:

- 1) равенство (`==`);
- 2) больше (`>`);
- 3) меньше (`<`);
- 4) не равно (`!=`);
- 5) больше или равно (`>=`);
- 6) меньше или равно (`<=`).

Операции сравнения

==

!=

>

>=

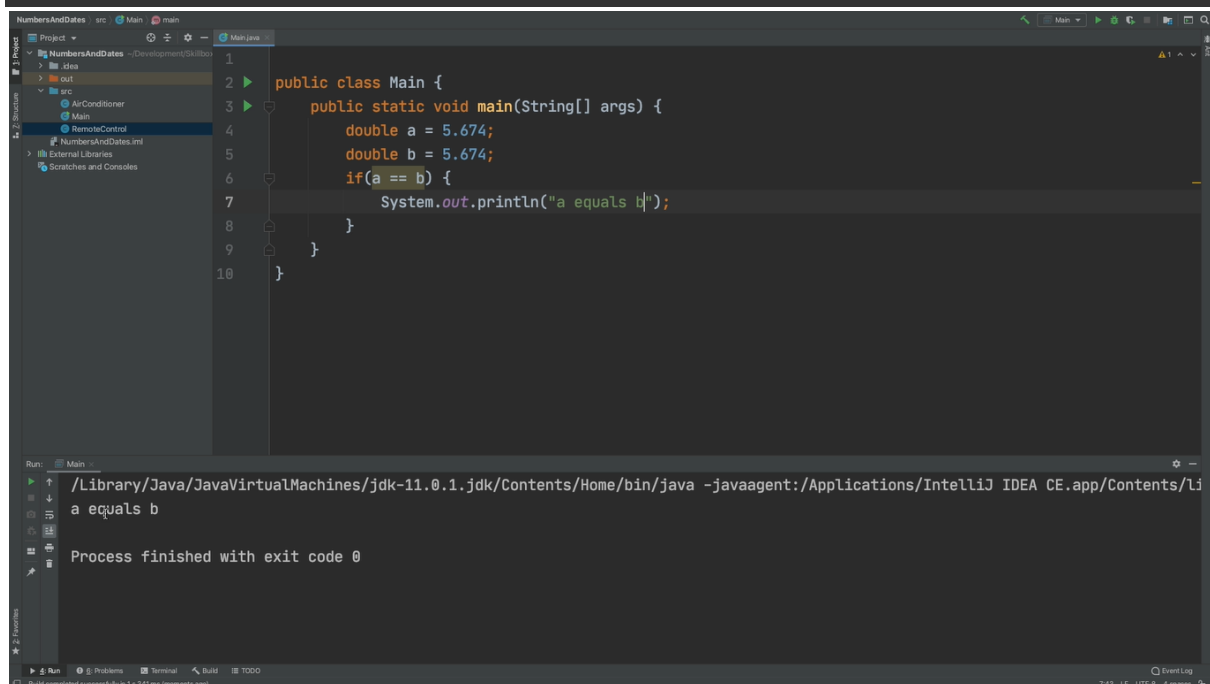
<

<=

Посмотрим, как работают эти операторы в коде.

Создадим два числа: a = 5,674 и b = 5,674. Дальше пишем условие:

```
if (a == b) {  
    System.out.println("a equals b");  
}
```



The screenshot shows the IntelliJ IDEA IDE. The main editor displays a Java class named `Main` with the following code:

```
1 public class Main {  
2     public static void main(String[] args) {  
3         double a = 5.674;  
4         double b = 5.674;  
5         if(a == b) {  
6             System.out.println("a equals b");  
7         }  
8     }  
9 }  
10
```

The Run window at the bottom shows the execution output:

```
Run: Main  
/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/Li  
a equals b  
Process finished with exit code 0
```

Сообщение выведено в консоль, всё работает верно.

Важный момент: в результате выполнения операции сравнения получается переменная типа `boolean`. При желании такое сравнение можно выделить в отдельную переменную, назвать её, например,

comparison (сравнение) и уже в новых условиях использовать этот элемент — результат будет тем же:



```
1 public class Main {
2     public static void main(String[] args) {
3         double a = 5.674;
4         double b = 5.674;
5         boolean comparison = a == b;
6         if (comparison) {
7             System.out.println("a equals b");
8         }
9     }
10 }
11 }
```

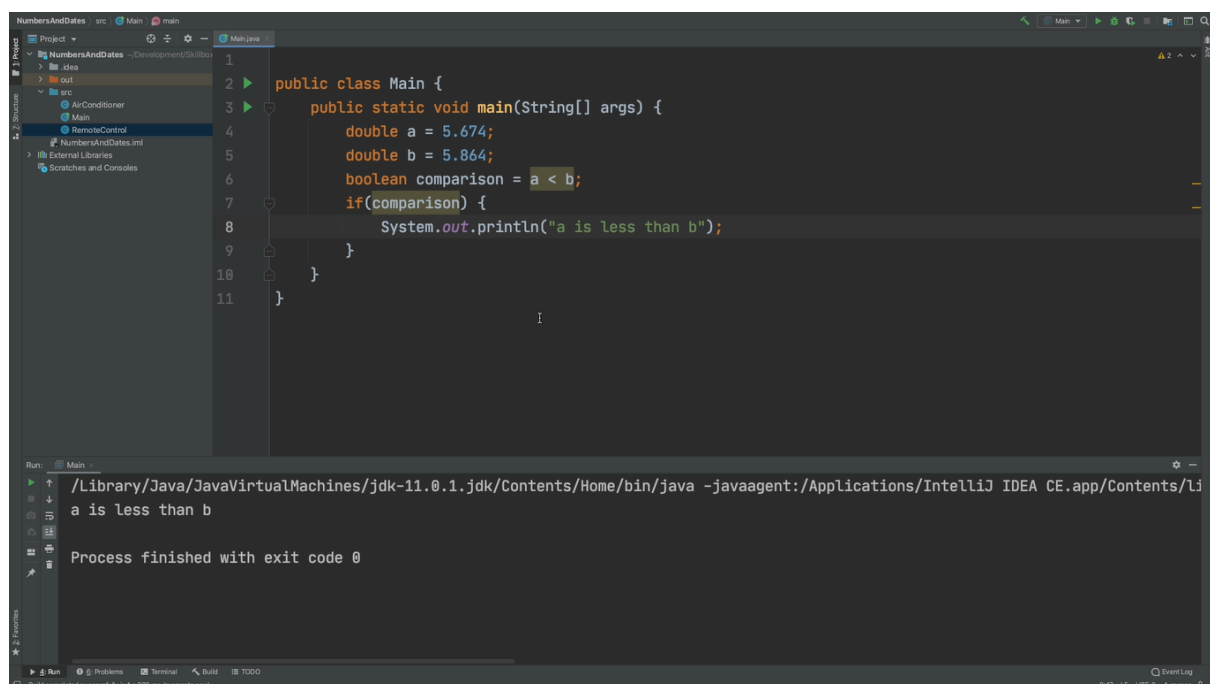
Run: Main

/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/Lib/idea-runtime.jar -jar /Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java

a equals b

Process finished with exit code 0

Это работает для всех операторов:



```
1 public class Main {
2     public static void main(String[] args) {
3         double a = 5.674;
4         double b = 5.864;
5         boolean comparison = a < b;
6         if (comparison) {
7             System.out.println("a is less than b");
8         }
9     }
10 }
11 }
```

Run: Main

/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/Lib/idea-runtime.jar -jar /Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java

a is less than b

Process finished with exit code 0

Выделение отдельных операций сравнения в переменную бывает полезно, если в операторе if формируется какой-нибудь очень большой, малопонятный, плохо читаемый код:

Операции сравнения

```
if(isTest && testsCount <=
    testsLimit &&
    SortedBox.getReleasesCount() >
    artPain.getMaxVersion()) {
    sendNotification(
        NOTIFICATION_TYPE.VERSION_OK
    );
}
```

Его можно преобразовать, например, в более понятный вариант:

Операции сравнения

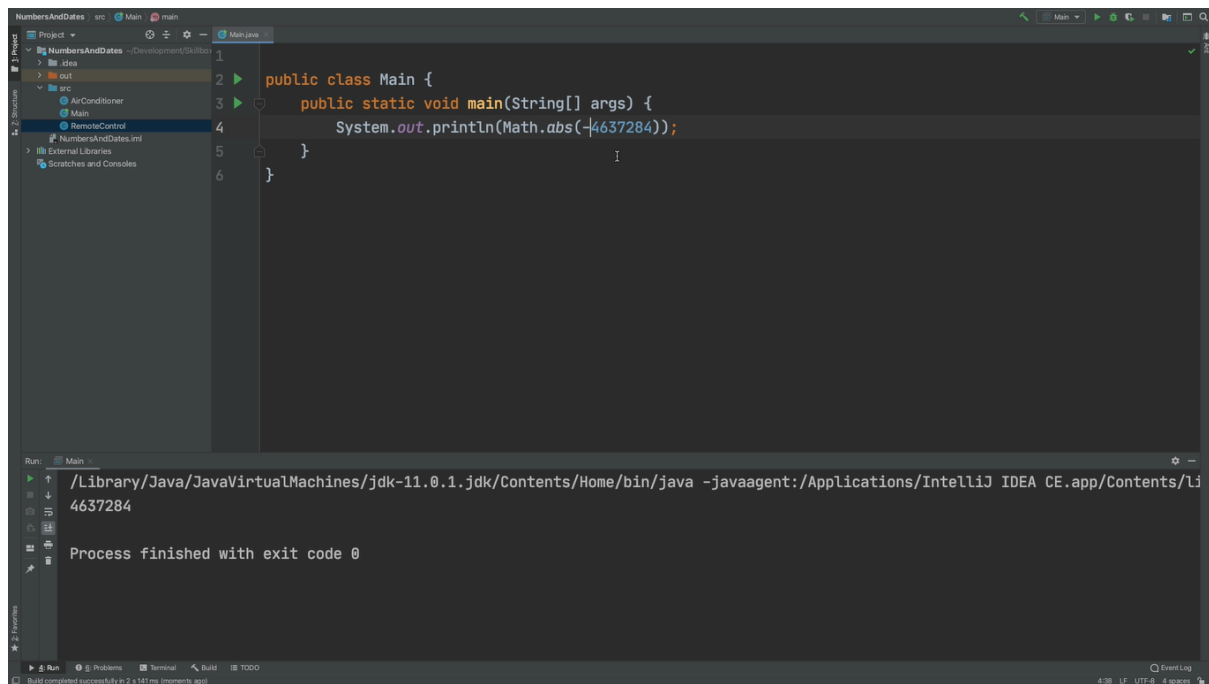
```
boolean testsCountIsOk = isTest &&
    testsCount <= testsLimit;
boolean versionIsOk =
    SortedBox.getReleasesCount() >
    artPain.getMaxVersion();
if(testsCountIsOk && versionIsOk) {
    sendNotification(
        NOTIFICATION_TYPE.VERSION_OK
    );
}
```

07:39–10:39

Класс Math

Поговорим о более сложных математических операциях. В Java есть специальный класс Math, который предоставляет множество методов для выполнения подобных операций.

Например, есть статический метод **abs**, который выдаёт абсолютное числовое значение числа по модулю:



Также у класса `Math` есть методы вычисления максимального и минимального значения — **max/min**. Есть по четыре вида таких методов, так как в них можно передавать числа `int`, `long`, `float`, `double`.

Кроме того, у этого класса есть методы, позволяющие выполнять тригонометрические функции: **sin**, **cos**, **tan**, **atan** и так далее.

Есть метод, позволяющий возводить число в степень, — **pow**. Например, число 5 возводим в квадрат и получаем число 25:

The screenshot shows the IntelliJ IDEA IDE with a project named 'NumbersAndDates'. The 'Main.java' file is open, containing the following code:

```
1 public class Main {  
2     public static void main(String[] args) {  
3         System.out.println(Math.pow(5, 2));  
4     }  
5 }  
6
```

The Run window at the bottom shows the command executed: `/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/Li` and the output: `25.0`. The process finished with exit code 0.

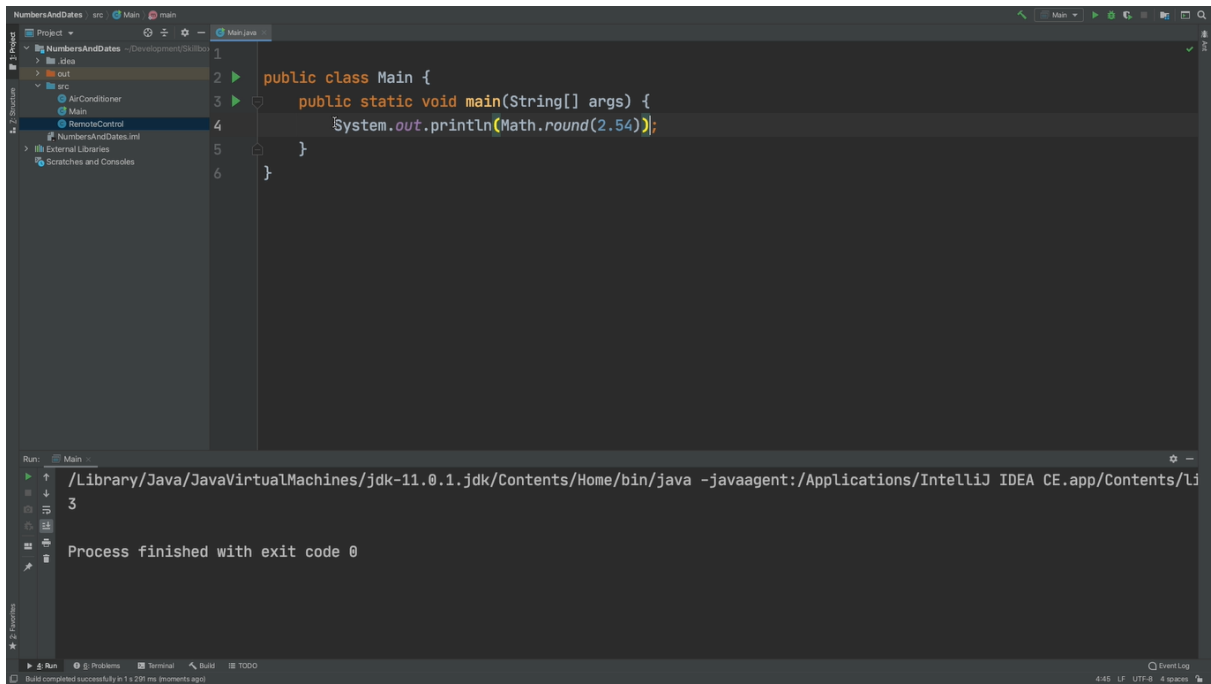
Есть метод, который позволяет вычислять квадратный корень, — **sqrt**:

The screenshot shows the IntelliJ IDEA IDE with the same project 'NumbersAndDates'. The 'Main.java' file is open, containing the following code:

```
1 public class Main {  
2     public static void main(String[] args) {  
3         System.out.println(Math.sqrt(625));  
4     }  
5 }  
6
```

The Run window at the bottom shows the same command as the previous screenshot, and the output is `25.0`. The process finished with exit code 0.

У класса **Math** есть очень полезный метод округления — **math.round**. Он округляет или число `float`, или число `double` до целого числа по всем математическим правилам округления.

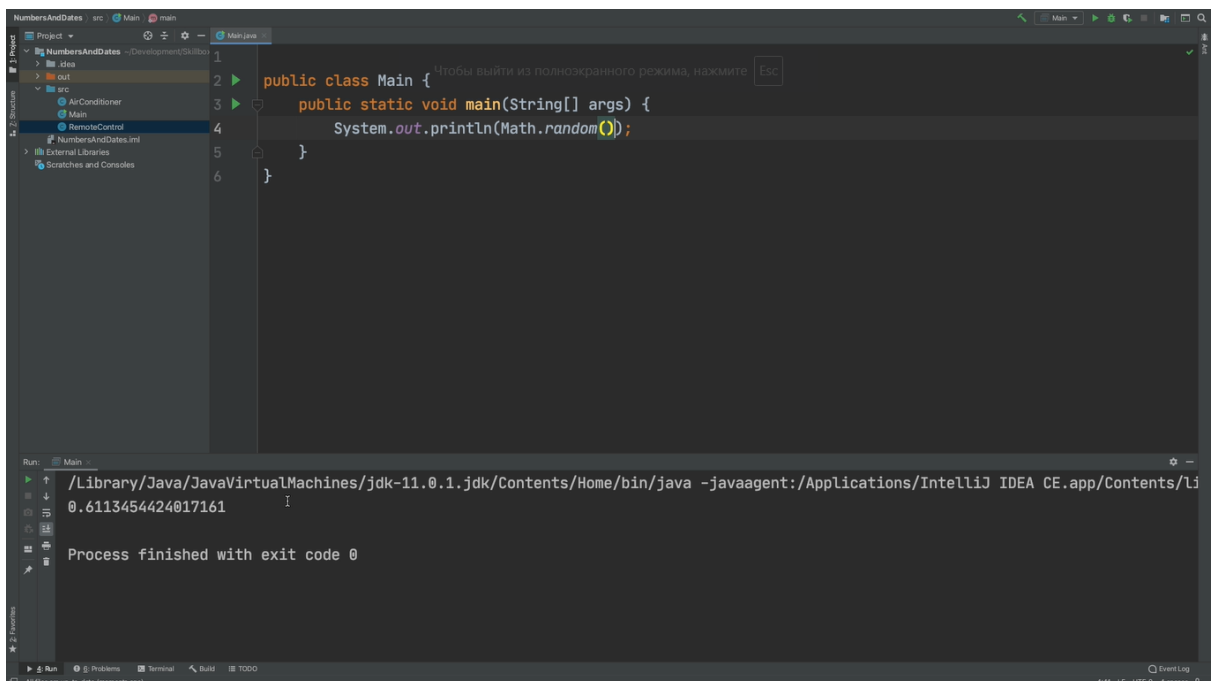


The screenshot shows the IntelliJ IDEA IDE with a project named 'NumbersAndDates'. The 'Main.java' file is open, containing the following code:

```
1 public class Main {
2     public static void main(String[] args) {
3         System.out.println(Math.round(2.54));
4     }
5 }
6
```

The 'Run' window at the bottom shows the command executed: `/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/Li`. The output is `3`, and the process finished with exit code 0.

У класса `Math` есть ещё один очень полезный метод, который позволяет генерировать случайные числа, — `math.random`. Этот метод генерирует случайные числа от 0 до 1, включая 0 и не включая 1.



The screenshot shows the IntelliJ IDEA IDE with the same project. The 'Main.java' file is open, containing the following code:

```
1 public class Main {
2     public static void main(String[] args) {
3         System.out.println(Math.random());
4     }
5 }
6
```

The 'Run' window at the bottom shows the command executed: `/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/Li`. The output is `0.6113454424017161`, and the process finished with exit code 0.

В Java есть и другие способы генерации случайных чисел, но пока ограничимся только `math.random`.

Повторим операции, с которыми вы только что познакомились:

- вычисление числа по модулю,
- вычисление минимального и максимального значения,
- тригонометрические функции,
- возведение в степень,
- извлечение квадратного корня,
- округление,
- генерация случайных чисел от нуля до единицы.

Класс Math

<code>Math.abs (...)</code>	<code>Math.pow (...)</code>
	<code>Math.sqrt (...)</code>
<code>Math.min (...)</code>	
<code>Math.max (...)</code>	<code>Math.PI</code>
	<code>Math.E</code>
<code>Math.sin (...)</code>	
<code>Math.cos (...)</code>	<code>Math.round (...)</code>
<code>Math.tan (...)</code>	<code>Math.random (...)</code>

10:39–11:09

Итоги

Итак, в этой теме вы познакомились с различными операциями, которые можно производить над числами:

- арифметическими операциями, в том числе с операциями получения остатка от деления;
- операциями с присвоением;
- операциями сравнения.

Также познакомились с методами класса `Math`, предоставляющими различные математические функции.

В следующей теме вы познакомитесь с ещё одним типом математических операций — операциями инкремента и декремента.